



■ CS177 Bioinformatics: Software Development and Applications

Lecture 3: Algorithms and Probability

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2025





Learning objectives



- At the end of this lecture, you will be able to:
 - Understand what algorithms are
 - Measure running time of an algorithm
 - Understand some common techniques for algorithm design
 - Define and use some basic concepts of probability (e.g. Bayes' theorem, maximum likelihood)





What Is Algorithm?





Algorithm



- An **algorithm** is a sequence of instructions that one must perform in order to solve a well-formulated **problem**
- Problems are specified in terms of their **inputs** and **outputs**
- The algorithm will be the method of translating the inputs into the outputs
- A well-formulated problem is unambiguous and precise, leaving no room for misinterpretation



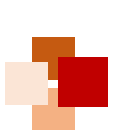


Pseudocode



- Using a programming language (e.g. C or Java) to describe an algorithm will contain too many low-level technical details
- **Pseudocode** is a language we often use to describe algorithms. It ignores many details required in a programming language, yet it is more precise and less ambiguous than using a natural language





Variables and arrays



- A **variable**, written as x or $total$, contains some numerical value and can be assigned a new numerical value at different points throughout the course of an algorithm
- An **array** of n elements is an ordered collection of n variables $a_1, a_2, \dots, a_n$
- Arrays are usually denoted by bold-face letters like $\mathbf{a} = (a_1, a_2, \dots, a_n)$ and the individual elements are written as a_i , where i is between 1 and n





Subroutines



- For clarity and modularity of coding, operations are often grouped into mini-algorithms called **subroutines** (or **functions**)
- An algorithm in pseudocode is denoted by a **name**, followed by a list of **arguments** / **parameters** that it requires, e.g. **Max(a, b)**, which is followed by the statements that describe the algorithm's actions
- One can **call** or **invoke** an algorithm by passing it the appropriate values for its arguments
- The operation of **return** reports the results of the algorithm or just signals its end





Basic operations



- Assignment: $a \leftarrow b$
- Arithmetic: $a + b$, $a - b$, $a \cdot b$, a/b , a^b
- Conditional: **if** (A is true) ... **else** ...
- For loops: **for** $i \leftarrow a$ to b {B}
- While loops: **while** (A is true) B
- Array access:
 - a_i denotes the i th number of array $\mathbf{a} = (a_1, a_2, \dots, a_n)$





Meaning of correct and incorrect algorithms



- We say that an algorithm is **correct**, if it can translate every input instance into the correct output
- An algorithm is **incorrect**, if there is at least one input instance for which the algorithm does not produce the correct output
- A critical but healthy pessimism: Unless you can justify that an algorithm **always** returns correct results, you should consider it to be wrong
- For some difficult problems (e.g. NP-complete problems, to be explained later), however, we rely on **approximation** algorithms, because no practical and correct algorithm can be found





Fast versus Slow Algorithms



Measure of running time



- Suppose our computer takes 10^{-9} second to perform an elementary operation, and we know how many operations an algorithm would perform, then the running time will be the product of **the number of operations** and **the time per operation**
- However, an algorithm could be run on different computers. Thus, the time per operation is not fixed
- Hence, rather than computing an algorithm's running time on every computer, we rely on the **total number of operations** that the algorithm performs to describe its running time
- As such, the running time will be an attribute of the algorithm, and not an attribute of a specific computer

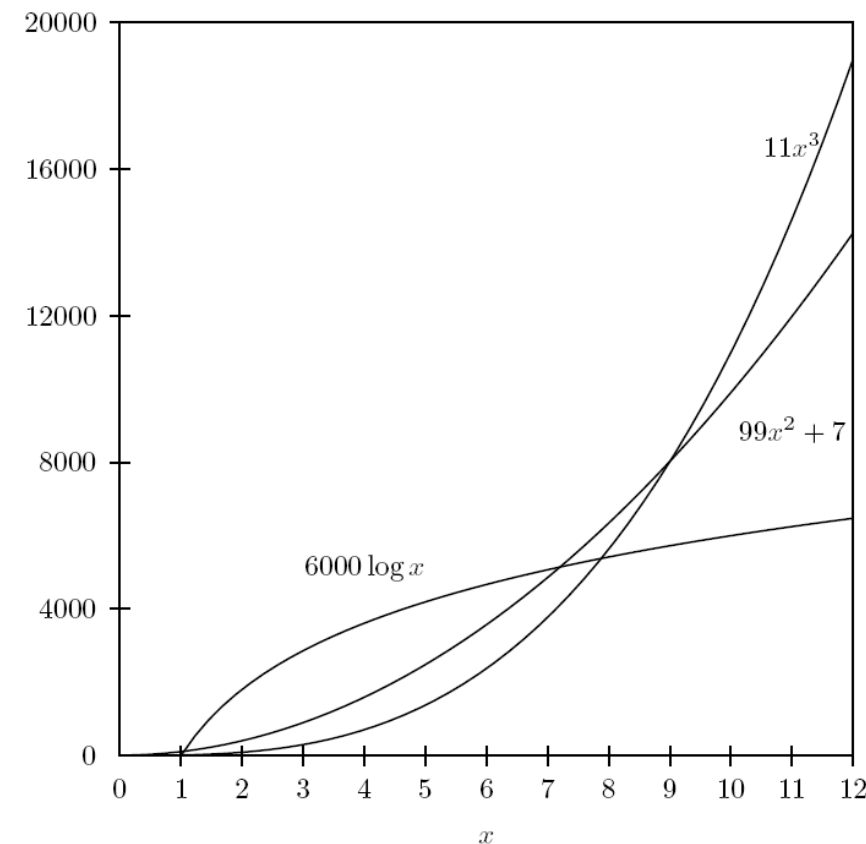




Running time as a function of input size



- Typically, the larger the input is, the longer the algorithm takes to process it
- To describe the performance of an algorithm for all kinds of input, and for comparison between algorithms, we need a high-level understanding of the **growth of an algorithm's operation count as the input size increases**
- Example: To solve a problem on an input of size n , algorithm A performs $11n^3$ operations on an input of size n , and algorithm B performs $99n^2 + 7$. Which algorithm is faster?
- For *large* n , B performs much less operations than A, so B is faster (i.e. **more efficient**)



A comparison of a **logarithmic** ($6000 \log x$), a **quadratic** ($99x^2 + 7$) and a **cubic** ($11x^3$) function.





Exponential versus polynomial



- An algorithm is called an **exponential** algorithm, if its running time includes a term like M^d , where d is a **parameter** of the problem (i.e. d is a **variable** and can be made arbitrarily large by changing the input)
- The running time of a **polynomial** algorithm is bounded by a term like M^k , where k is a **constant** not related to the size of any parameter
- Example: A polynomial algorithm with running time M^1 (**linear**), M^2 (**quadratic**), M^3 (**cubic**), or even M^{2000}





Best, worst and average cases



- **Best-case analysis** gives us the **minimum** number of primitive operations performed by the algorithm on any input of size n (i.e. best-case time complexity)
- **Worst-case analysis** gives us the **maximum** number of primitive operations performed by the algorithm on any input of size n (i.e. worst-case time complexity)
- **Average-case analysis** gives us the **average** number of primitive operations performed by the algorithm on all inputs of size n
- Note:
 - Best case does not mean that n is small (e.g. $n = 1$), because the running time is represented as a function of n , where n is not fixed
 - For some problems, the input size may include multiple parameters

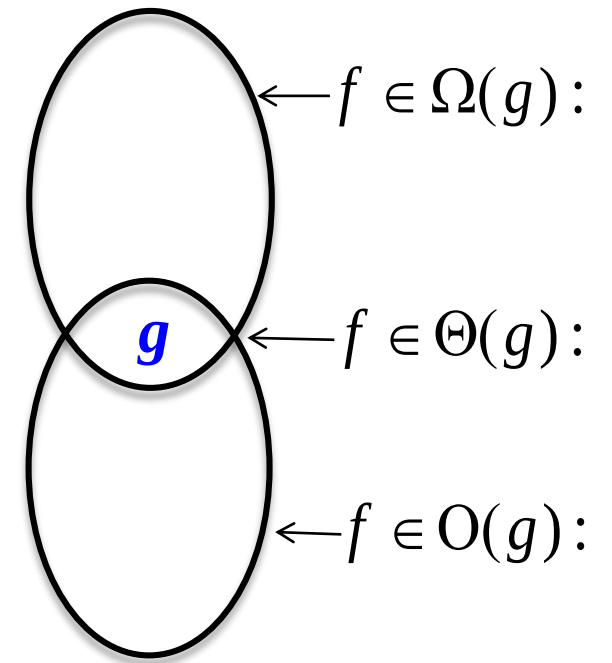




Big-O notation



- Computer scientists use the Big-O notation to describe the running time of an algorithm **concisely**.
- All functions with a leading terms of n^2 have more or less the same rate of growth, e.g. $99.7n^2$, $0.001n^2$ or $5n^2 + 3.2n + 99993$. We lump them into one class, which we call $O(n^2)$.
- We ignore the leading constant and pay attention only to the **fastest-growing term**.
- Big-Oh (O), Big-Omega (Ω) and Big-Theta (Θ) are **asymptotic** (set) notations used for describing the **order of growth** of a given function.





Algorithm Design Techniques





Problem of searching for a phone



- There are relatively few basic techniques for algorithm design
- To illustrate the techniques, we consider a simple problem:
 - Suppose your mobile phone rings, but you have misplaced it somewhere in your home. How do you find it?
 - To complicate matters, you have just walked into your home with an armful of groceries, and it is dark so you cannot rely solely on eyesight





Exhaustive search



- An **exhaustive search**, or **brute force**, algorithm examines every possible alternative to find one particular solution
- To find the ringing phone, you would ignore the ringing as if you could not hear it, and simply walk over every single square inch of your home
- You would be guaranteed to find the phone no matter where it is
- They are **easy to design and understand**, but often **too slow** to be practical





Branch-and-Bound



- As we explore the various alternatives in a brute force algorithm, we may discover that a large number of alternatives can be skipped. This technique is called **branch-and-bound**, or **pruning**
- While exhaustively searching the first floor, if you hear the phone ringing **above** your head, you could immediately rule out the need to search the basement or the first floor
- What might have taken 3 hours may now only take 1 hour

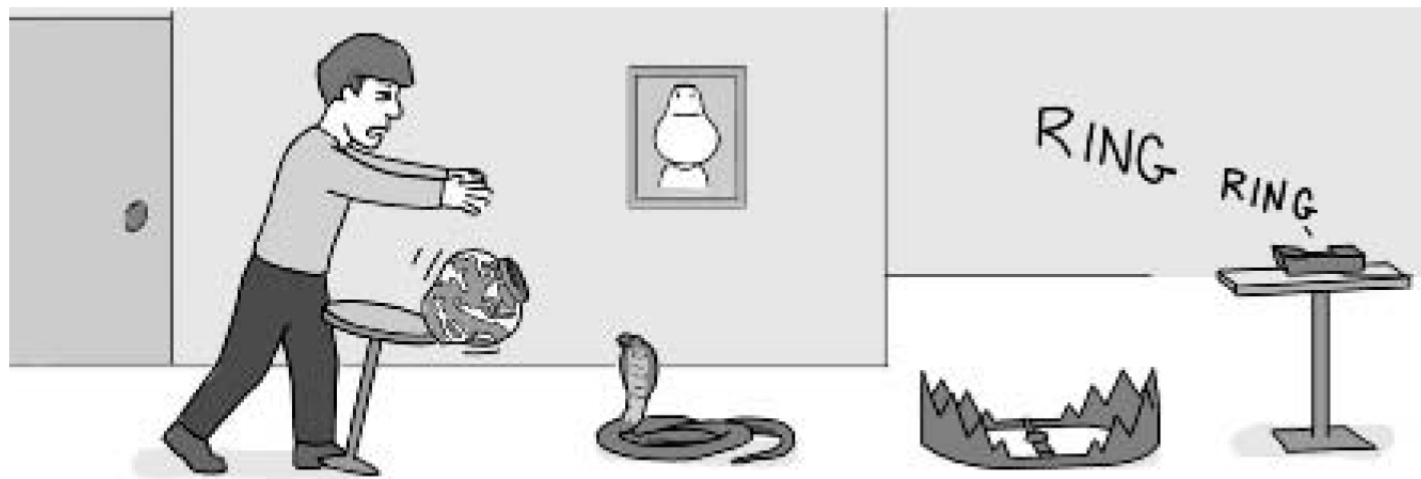




Greedy algorithms



- Many algorithms are iterative procedures that choose among a number of alternatives at each iteration
- **Greedy algorithms** choose the “most attractive” alternative at each iteration.
- Example (Phone searching problem): the greedy algorithm would simply be to walk in the direction of the phone's ringing until you find it. However, there may be a **wall** (or an expensive and fragile **vase**) between you and the phone, preventing you from finding it



- One big problem may be hard to solve, but two problems of half the size may be much easier
- Steps of a divide-and-conquer algorithm:
 - Split the problem into smaller subproblems
 - Solve the subproblems independently
 - Combine the solutions of the subproblems into a solution of the original problem
- Note: The algorithm usually splits the subproblems into even smaller sub-subproblems, **recursively**, until it reaches the terminating condition



Dynamic programming



- Some algorithms break a problem into smaller subproblems (e.g. divide-and-conquer). However, the number of subproblems might become very large, and some algorithms solve the same subproblem repeatedly, wasting running time
- Divide-and-conquer is suitable when the subproblems are **independent** of each other. By contrast, dynamic programming is applicable when the subproblems are not independent, that is, when the subproblems share sub-subproblems
- **Rationale of dynamic programming:** Organize computations to avoid recomputing values that you already know, which can save a lot of time
- To avoid recomputing, dynamic programming saves the answer of a sub-subproblem in a **table**, which is looked up when the answer is needed





Machine learning



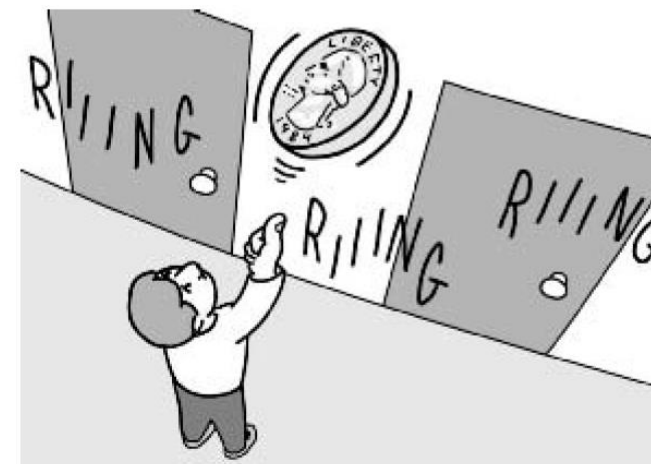
- A machine learning solution to the phone search problem:
 - Collect statistics over the course of a year about where you leave the phone, learning where the phone tends to end up most of the time
 - Suppose the phone was left in the bathroom 80% of the time, in the bedroom 15% of the time, and in the kitchen 5% of the time
 - Then, a sensible time-saving strategy is to start the search in the bathroom, then the bedroom, and finish in the kitchen
- The performance of a machine learning algorithm often depends on the previously collected data
- The close relation with data analysis makes machine learning methods suitable for bioinformatics



■ Randomized algorithms



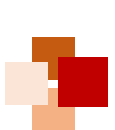
- Before starting to search for the phone, you could toss a coin to decide:
 - If the coin comes up heads, you start the search on the first floor
 - If the coin comes up tails, you start on the second floor
- Sometimes, randomized algorithms do have competitive advantage over deterministic algorithms





Basics of Probability





Sample space and frequency



- **Sample Space** (Ω) of an experiment is the non-empty set of all the mutually exclusive outcomes, e.g.
 - Tossing a coin: $\Omega = \{H, T\}$
 - Rolling a six-face die: $\Omega = \{1, 2, 3, 4, 5, 6\}$
 - Randomly selecting a base from DNA sequence: $\Omega = \{A, C, G, T\}$
- **Frequency of an event** – if an experiment is done n time and an event (the outcome) e occurred $n(e)$ times, then the frequency $f(e)$ of that event occurred in the experiment is

$$f(e) = \frac{n(e)}{n}$$





Probability

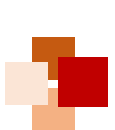


- **Events** in nature are deterministic, probabilistic, or fuzzy. The probability is associated with a *stochastic (probabilistic)* event.
- If an experiment is performed infinite times, the probability of an event is equal to the frequency of that event:

$$P(e) = \lim_{n \rightarrow \infty} \frac{n(e)}{n}$$

- The outcome of a probabilistic event is uncertain and can be stated only with a probability
- A **probability** P of an event e on the sample space Ω is a mapping from Ω to the unit interval $[0, 1]$, $P: \Omega \rightarrow [0, 1]$, which satisfies:
 1. $P(e) \geq 0$;
 2. $P(\Omega) = 1.0$;
 3. $\sum_{e \in \Omega} P(e) = 1.0$ for mutually exclusive and exhaustive events e in the sample space Ω





Conditional probability



- **Joint probability** $P(e_1, e_2)$ is the probability that two (or more) events, say e_1 and e_2 , have happened together and is given by the probability of the intersection of two events.
- **Conditional probability** $P(e_1 | e_2)$ of two events, say e_1 given e_2 , is the probability of event e_1 given that the event e_2 (the *prior* event) has already occurred, and is given by

$$P(e_1 | e_2) = \frac{P(e_1, e_2)}{P(e_2)}$$

- Therefore, $P(e_1, e_2) = P(e_1 | e_2)P(e_2)$
- **Independent events:** The probability of one event does not depend on the other event. Therefore, $P(e_1 | e_2) = P(e_1)$ and $P(e_2 | e_1) = P(e_2)$
- That is, the joint probability of independent events is equal to the product of their individual probabilities:

$$P(e_1, e_2) = P(e_1)P(e_2)$$



Bayes' Theorem

- For two events e_1 and e_2 , since $P(e_1, e_2) = P(e_1 | e_2)P(e_2) = P(e_2 | e_1)P(e_1)$,

Bayes' theorem is given by

$$P(e_1 | e_2) = \frac{P(e_2 | e_1)P(e_1)}{P(e_2)}$$

- For a given set of mutually exclusive and exhaustive event $e, e_1, e_2, \dots, e_n$, the probability of event e is give by **the total probability formula (marginalization)**:

$$P(e) = \sum_{i=1}^n P(e, e_i) = \sum_{i=1}^n P(e | e_i)P(e_i)$$

- For such events, the Bayes' theorem is extended to

$$P(e_i | e) = \frac{P(e | e_i)P(e_i)}{P(e)} = \frac{P(e | e_i)P(e_i)}{\sum_{j=1}^n P(e | e_j)P(e_j)}$$



Random variable



- A **random variable** X is defined as a function $X: \Omega \rightarrow D$ on the probability space (Ω, P) . The variable is random because the value of the variable is determined probabilistically by P .
- $P(X = x)$ denotes that the probability of the random variable X taking the value x , where $x \in D$.
- Random variables may be:
 - **Discrete**, i.e. the variable is defined on a set of discrete points, or
 - **Continuous**, i.e. the variable is defined on a continuous domain





■ Probability mass function (pmf)

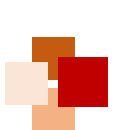
- For a discrete random variable X taking distinct values, $x_1, x_2, \dots, x_n$ the **probability mass function**, denoted by p_X , is defined by

$$p_X(x) = \begin{cases} P(X = x_i), & i = 1, 2, \dots, n \\ 0, & \text{otherwise.} \end{cases}$$

and $\sum_{i=1}^n p_X(x_i) = 1$

- Notation:
 - Upper-case P denotes a value of probability;
 - Lower-case p denotes a probability distribution;
 - Upper-case X denotes the random variable;
 - Lower-case x denotes the value of random variable.





Probability density function (pdf)



- For a continuous random variable, the probability of a given random variable $X \in \mathbf{R}$ is given by a **probability density function**, $p_X : \mathbf{R} \rightarrow [0, 1]$ such that

$$P(a \leq x \leq b) = \int_a^b p_X(x) dx$$

where $a, b \in \mathbf{R}$

- Note: $\int_{-\infty}^{+\infty} p_X(x) dx = 1$ and the probability of a continuous random variable is defined on a **range** of its values but not at **a particular value**





Expectation



- Let X be a random variable and $g: \Omega \rightarrow \mathbb{R}$ be a real-valued function. The **expectation** or **expected value** of the function g with a random variable X , is defined as:

$$E(g(X)) = \begin{cases} \sum_{x_i \in \Omega} g(x_i) p_X(x_i), & \text{for discrete variables;} \\ \int_{-\infty}^{+\infty} g(x) p_X(x) dx, & \text{for continuous variables.} \end{cases}$$

- Mean** is the expectation of random variable X , $E[X]$, and is defined as:

$$\mu_X = \begin{cases} \sum_{x_i \in \Omega} x_i p_X(x_i), & \text{for discrete variables;} \\ \int_{-\infty}^{+\infty} x p_X(x) dx, & \text{for continuous variables.} \end{cases}$$

- Moments**: If X is a random variable, the **r th moment of X** , denoted by m'_r , is given by $m'_r = E[x^r]$. Note that

$$m'_1 = \mu_X$$





Variance



- **Central moments** : If X is a random variable, the **r th central moment** of X **about a** is defined as $E[(X-a)^r]$. If $a = \mu_X$, we have the r th central moment about the mean μ_X , which is denoted by m_r :

$$m_r = E[(X - \mu_X)^r]$$

- **Variance** is the 2nd central moment of a variable about its mean, denoted by $\text{var}[]$:

$$\text{var}[X] = m_2 = E[(X - \mu_X)^2]$$

- **Standard deviation (SD)**, σ , is the square root of the variance:

$$\sigma_X = (\text{var}[X])^{1/2}$$





Correlation



- **Covariance** : Let X and Y be two random variables defined on the same probability space. Then the **covariance** of X and Y , denoted by $\text{cov}[X, Y]$, is defined as

$$\text{cov}[X, Y] = E[(X - \mu_X)(Y - \mu_Y)]$$

- **Correlation coefficient** is denoted by $\rho[X, Y]$ of two random variables X and Y . It is defined as

$$\rho[X, Y] = \frac{\text{cov}[X, Y]}{\sigma_X \sigma_Y}$$

where σ_X and σ_Y are the SD of X and Y , respectively.

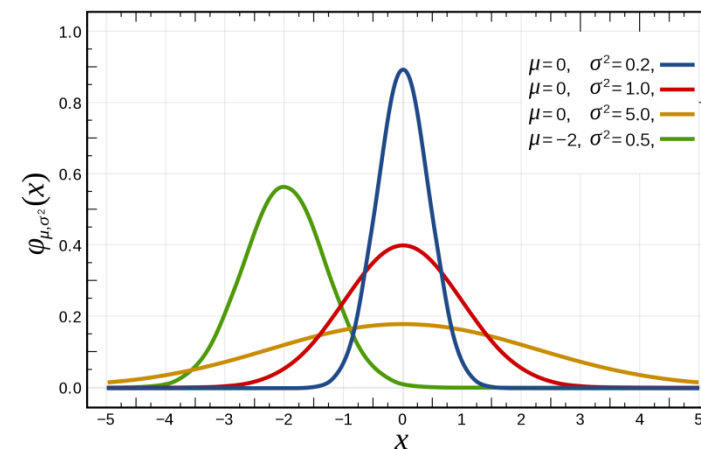




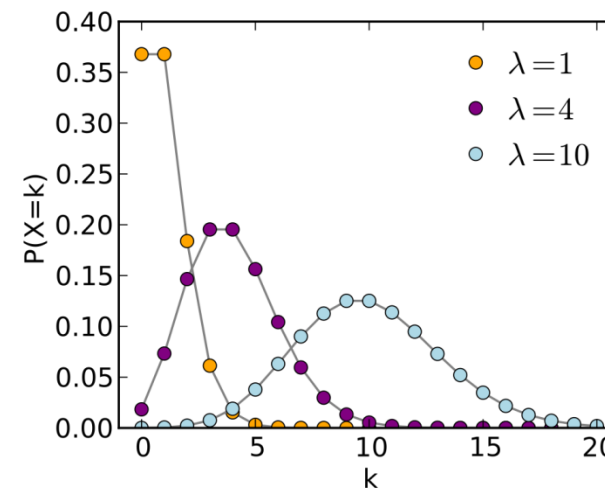
Common probability distributions



- Uniform distribution
- Bernoulli distribution
- Binomial distribution
- Multinomial distribution
- Normal (Gaussian) distribution
- Geometric distribution
- Poisson distribution
- Dirichlet distribution
- Gamma distribution
- Chi-square (χ^2) distribution
- etc.



The normal (Gaussian) distribution



The Poisson distribution





Maximum Likelihood



- Most biological phenomena are due to random events or interpreted with probability
- How to build probabilistic models for biological data, processes, or phenomena?
 - E.g. sequences (DNA, protein, etc.), gene expressions, protein structures, evolution
- Such a model assigns **high probability** to data/information when it fits the phenomenon well, and **low probability** for those it does not fit well
- Issues in probabilistic modeling:
 - What is the best model? Multiple models; methods to assess how well a given dataset D fits a model instance M ; model selection problem
 - What is the learning algorithm? That is, how to determine the parameters of the model?
 - Amount of data available for model training (or parameter estimation)?
 - Characteristics of data (noise, independence, redundancies, biases)?
 - Unless stated otherwise, the data $d \in D$ are generally assumed to be independent





Maximum Likelihood (ML)



- Once the model M is chosen, the parameters of the model have to be inferred from the data. This is referred to as **learning** or **training** of the model
- Given the model M and its parameters α , the **likelihood** of data D is given by

$$P(D | \alpha, M)$$

- The likelihood indicates how well the model predicts the data
- The **maximum likelihood (ML)** estimator maximizes the likelihood of the data given the model. That is, it finds the optimal set of parameters α^{ML} that maximize the likelihood:

$$\alpha^{\text{ML}} = \arg \max_{\alpha} P(D | \alpha, M)$$

- Often the log-likelihood (natural logarithm of the likelihood function) is used for computational efficiency.





Strengths and drawback of ML



- **Consistency**: Maximum likelihood is **consistent** in the sense that the true (unknown) parameter α_0 will also, in the limit of a large amount of data, be the value that maximizes the likelihood, i.e., $\alpha \rightarrow \alpha_0$ as the data size $n \rightarrow \infty$
- Other nice properties of ML:
 - **Efficient**: needs less data than other estimators to achieve a given performance
 - **Invariance to parameter transformation**: If θ^* is the MLE of θ , then for any function $f(\theta)$, the MLE of $f(\theta)$ is $f(\theta^*)$
- Drawback: When the data are scanty, ML can give poor results
 - Example: In rolling a die, to estimate the probabilities of the 6 faces, $\theta_1, \theta_2, \dots, \theta_6$, if we use only 3 different rolls of the die, then the ML estimate is
$$\theta_i = n_i / \sum n_k$$
 - But then, at least 3 of the 6 parameters have values 0, a bad estimator
 - Solution: To incorporate *prior* knowledge (e.g. $\theta_1, \theta_2, \dots, \theta_6$ should all be near 1/6)



Maximum a posteriori (MAP)

- Given the data D and the model M , the **posterior probability** is the probability of parameters, $P(\alpha | D, M)$
- From Bayes' theorem:

$$P(\alpha | D, M) = \frac{P(D | \alpha, M)P(\alpha, M)}{P(D, M)} = \frac{P(D | \alpha, M)P(\alpha | M)P(M)}{P(D, M)}$$

- Because the parameters α do not depend on the terms $P(M)$ or $P(D, M)$,
$$P(\alpha | D, M) \propto P(D | \alpha, M)P(\alpha | M)$$

- The **maximum a posteriori (MAP)** estimator gives the parameters that maximize the posterior probability of the parameters, i.e. the MAP estimator is given by

$$\alpha^{\text{MAP}} = \arg \max_{\alpha} P(D | \alpha, M)P(\alpha | M)$$

- The **prior probability** of parameters $P(\alpha | M)$ is chosen in some reasonable manner to incorporate *prior* (biological) knowledge (Bayesian statistics)





Recap



- In this lecture, we have learned:
 - What is an algorithm
 - How to use pseudocode to describe algorithms
 - How to measure the running time of algorithms
 - Common techniques of algorithm design
 - Basics of probability theory and probabilistic modeling





References



- N. Jones and P. Pevzner. "Algorithms and Complexity" , Chapter 2 of their book *"An Introduction to Bioinformatics Algorithms"* . MIT Press, 2004.
- R. Durbin, S. Eddy, A. Krogh, G. Mitchison. "Biological sequence analysis: Probabilistic models of proteins and nucleic acids" , Chapter 3 "Markov chains and hidden Markov models" , Cambridge University Press, 1998.
- P. Baldi, S. Brunak. "Bioinformatics: The machine learning approach" , MIT Press, 2001.

